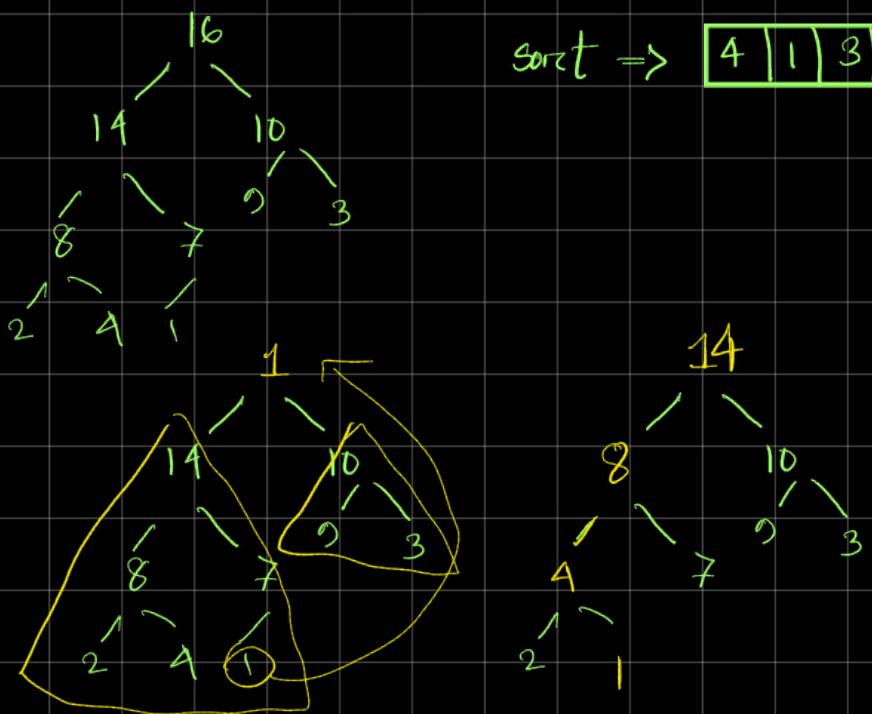
heapsort → O(nlogn)
→ in place , O(1)

insertion sort → $O(n^2)$ → (bubble sort ka) : in place sort. (done within input array)

merge " → O(nlogn) : extra place needed.

for inplace algorithm, space complx O(1)




sort => | 4 | 1 | 3 | 2 | 16 | 9 | 10 | 14 | 8 | 7 |

pop current max 16

take last number of the heap and put it in root

* Delete root
* Fix location

Extract max

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |

heapsize 10 ↑ last location

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 | 16 |

heap size = 9 ↓ storing it at the end of my array

Heap-sort(Algorithm)

build-max heap(A)

for i = length ..... 2 (can't swap when only 1 left)

swap(A[i], A[1]) → with last element

heapsize--

max_heapify(A,1)

| 14 | 16 |
|---|---|
| 8 | |

| 7 | 4 | 3 | 1 | 2 | 8 | 9 | 10 | 14 | 16 |
|---|---|---|---|---|---|---|---|---|---|

sorted

max heap → ascending order sort

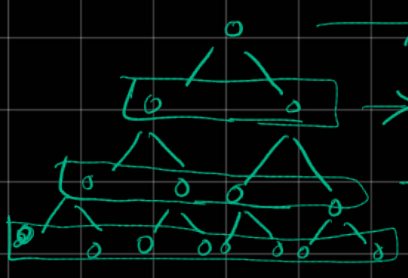
min " → descending " "

build max heap (A) → $O(n \log n) \rightarrow O(n)$

heapsize

for i = length/2 down to 1 → $n/2$

max heapify (A, i) → $O(\log n)$




(call const operation only once)

Time required:

$$T(n) = \frac{n}{4}(1c) + \frac{n}{8}(2c) + \frac{n}{16}(3c) \cdots\cdots + 1(c \log n)$$

$$= c\frac{n}{4}\left\{1 + \frac{2}{2} + \frac{3}{4} + \frac{4}{8} + \cdots\cdots \frac{\log n}{n/4}\right\}$$

$$= c\frac{n}{4}\left\{\frac{1}{2^0} + \frac{2}{2^1} \cdots\cdots \frac{\log n}{n/4}\right\}$$

$$= c \cdot 2^k \left[\left\{\frac{1}{2^0} + \frac{2}{2^1} + \cdots \frac{k+2}{2^k}\right\}\right]$$

bounded by const c'

decreasing functions

let $n/4 = 2^k$

$\log n$

$\log_2 (2^k \cdot 4) = \log_2 (2^{k+2})$

$= k+2$

$$T(n) = c \cdot 2^k \cdot c' = c'' \frac{n}{4} = O(N)$$

Priority Queue → (x, key)

max / min

- insert
- max / min
- extract max/min
- increase key

re-initiate heapsize back to original after heapsort

Max-PQ → heap

=> heap-maximum(A) — O(1)
return A[1] (take root from the heap)

=> Extract_max(A) — logn (heapify)
→ swap with last element
if heapsize < 1 → underflow)

```
max = A[1]
swap (A[1], A[heap_size])   // last element
heapsize--
maxheapify (A, 1)
return max
```

complexity → logn

insert 15
=> increase heapsize & insert at end ✗

compare with the parent node

# max Heap_insert (A, key)

```
heap_size++                 [i = heapsize]
A[heap_size] = key
```

while A[i] > A[parent of i] & (i > 1) do it until
swap(A[i], A[parent of i])
i = i/2

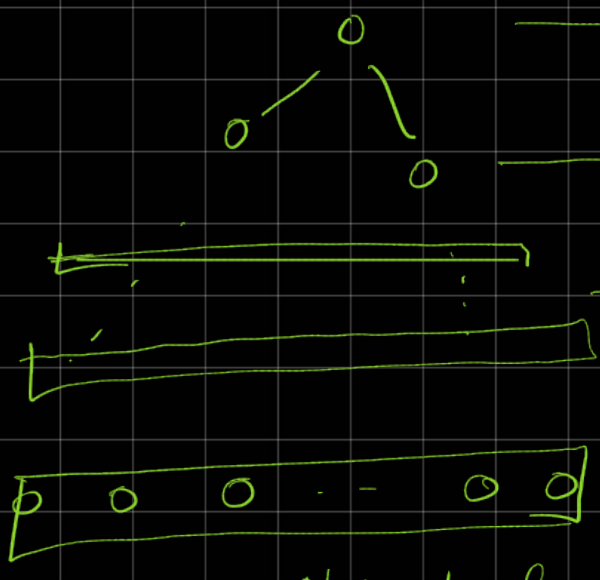
max Heap-increase (update a location)

$\log N$ => same as insert (compare with parent & place in proper position

Heap_increase_key(A, i, key)

priority Queue (key, value)

Build_maxheap() ⟶ $O(N \log N) \to O(N)$




$\log N$ (height No. of) comparisons

2C

1C major comparisons

N total

X → no comparison

$$T(N) = \frac{N}{2}0 + \frac{N}{4}1C + \frac{N}{8}2C \cdots + 1(\log N)C$$

$$= \frac{N}{4}C\left(1 + \frac{2}{2} + \frac{3}{4} + \frac{4}{8} \cdots \frac{\log N}{N/4}\right)$$

$$= \frac{N}{4}C\left(\frac{1}{2^0} + \frac{2}{2^1} + \frac{3}{2^2} + \cdots\right)$$

$2^0 \to 1$, $2^1 \to 1$, $2^2 \to 0.75$

$$\left(\frac{N}{4} = 2^K,\ N = 2^{K+2}\right) = C \cdot 2^K \cdot C' \quad (C' = C)$$

$1+1+\cdots \downarrow\downarrow < 4$

$T(N) \rightarrow O\left(\frac{N}{4}\right) \Rightarrow \sim O(N)$

max $\rightarrow$ $\log N$ swapping $\rightarrow$ increase